run_config.yaml

TEM_UW_APP run configuration, key reference

Lehigh TEM Felzer Lab

November 2026

Contents

1 Overview **1**

2 I/O and storm patch **2**

0. Job environment **2**

1. Job submission **2**

2. Case metadata **3**

3. Grid and time **3**

4. NetCDF inputs and varmap **3**

5. Model options **4**

6. Parameter files **5**

7. Output and model state **5**

1 Overview

`config/run_config.yaml` is the run configuration of TEM_UW_APP. A case-local `run_config.yaml` sets only the keys it changes; unknown keys are an error. The sections below are the file itself. Keys of the section Model options are written to `tem_config.yaml` of TEM_2026, whose template describes them in more detail.

2 I/O and storm patch

Apply `TEM_UW_APP-io.patch` with `git am` from this repository.

The case launcher stages NetCDF and writes the model YAML directly, including the output-variable sequences. Legacy ASCII and XML conversion tools are removed. Source adapters still ingest public data for the NetCDF builders.

Stochastic tropical storm and hurricane damage requires a TEM_2026 build with `STORM=1`, the default. Use `make STORM=0` to build the core without the feature. Set the following keys in `run_config.yaml` to enable it.

```yaml
storm_disturbance: true
netcdf_storm_in_file: /data/storm.nc
```

NetCDF variable	Dimensions	Units
`lat`	`lat`	`degrees_north`
`lon`	`lon`	`degrees_east`
`storm_return_interval`	`lat, lon`	`years`
`hurricane_return_interval`	`lat, lon`	`years`

Use one static NetCDF dataset with both return-interval fields on the same rectilinear grid. Zero disables the corresponding event. Positive intervals must be finite and at least one year. The model reads NetCDF content regardless of the filename extension. Generated files default to `.nc`.

The app regrids and stages the dataset as `INPUT/storm.nc`.

0. Job environment

```yaml
tem_src: null # TEM_2026 checkout: templates/tem_config.yaml, params/ and the build (REQUIRED)
# Executable built with make (build/xtem_competition for cohort competition); for n_cpus > 1
# use the MPI build, {tem_src}/build_mpi/xtem
tem_exe: "{tem_src}/build/xtem"
modules: [] # Modules loaded before the run (the libraries the executable was built with)
```

1. Job submission

```yaml
# Parameters forwarded to sbatch by launchers/case_submit.py, as in pyUFS:
#
#   key              sbatch flag
#   constraint_node  --constraint=<value>   (omitted when null)
#   exclusive_node   --exclusive            (omitted when false)
#   walltime         --time=<walltime>:00:00
#   n_nodes          --nodes
#   n_cpus           --ntasks and --ntasks-per-node
#   n_cpus_per_task  --cpus-per-task
#   partition        --partition            (omitted when null)
#   mem              --mem=<mem>g on one node, --mem-per-cpu=<mem // tasks>g on several
#   sbatch_options   appended to the sbatch call unchanged
#
# Tasks per node are n_cpus // n_nodes and the task count submitted is (n_cpus // n_nodes) *
# n_nodes. The model runs on that many MPI tasks; the cells are shared between them.

run_mode: local # local: build and run in the foreground | slurm: submit with sbatch
run_model: true # false: only build the case (INPUT/ and tem_config.yaml)
# Node constraint passed as --constraint=<value>, for example a CPU architecture or feature
# tag defined by the host
constraint_node: null
exclusive_node: false # true requests exclusive node access (--exclusive)
walltime: 1 # Walltime in whole hours, submitted as <walltime>:00:00
n_nodes: 1 # Number of nodes to request
n_cpus: 1 # Total number of tasks to request, rounded down to a multiple of n_nodes
partition: null # Partition to submit the job to
```

```yaml
n_cpus_per_task: 1 # CPUs per task
# Total job memory [GB]. Used only when above 2 GB per task (mem > 2 * tasks); otherwise, and
# for 0, the scheduler default applies.
mem: 0
# Additional raw options passed directly to sbatch, for example "--account=myproj --mail-
# type=END"
sbatch_options: ""
logfile: tem_driver # Basename of the driver log written in the case directory (<logfile>.log)
```

2. Case metadata

```yaml
case_name: null # Case identifier and SLURM job name; null falls back to the directory name
region: TEM # Label written to the output files
```

3. Grid and time

```yaml
horizontal_resolution: 0.5 # degrees
lon_min: null # REQUIRED
lon_max: null # REQUIRED
lat_min: null # REQUIRED
lat_max: null # REQUIRED
# box_mean (area average, when the source is finer) | bilinear | nearest. Categorical fields
# always use nearest.
regrid_method: box_mean
run_dt: 1 # Model time step, 1
# month | day. day needs daily forcing (netcdf_climate_in_frequency: daily) and a build with
# room for 365 steps (make daily)
run_dt_units: month
start_year: null # First year of the transient run (REQUIRED)
end_year: null # Last year of the transient run (REQUIRED)
```

4. NetCDF inputs and varmap

```yaml
# Each key is a path, a glob or a list of them (null: not used). The files are regridded to
# the grid above and written to INPUT/ as climate.nc, static.nc, lulc.nc, co2.nc, lightning.nc
# and population.nc. Floats are written as float32 and categorical fields as int64.

# Climate (REQUIRED), merged by coordinates; units come from the "units" attribute
netcdf_climate_in_file: null
# hourly | daily | monthly, the time step of netcdf_climate_in_file
netcdf_climate_in_frequency: monthly
# elev, sand, silt, clay, ndep (lat, lon); wind, ozone (month, lat, lon)
netcdf_static_in_file: null
netcdf_lulc_in_file: null # Land cover, (cohort, lat, lon); the fields of TEM_2026 docs/data.md
netcdf_co2_in_file: null # co2 (time), annual
# lightning_flashes, km-2 month-1 (12 months, lat, lon); needed when veg_fire is 1
netcdf_fire_in_file: null
netcdf_population_in_file: null # population_density, km-2 (lat, lon); needed when veg_fire is 1
# NetCDF return-interval grids in years, needed when storm_disturbance is true
netcdf_storm_in_file: null
surface_pressure_hpa: 1013.25 # Used for vpr from huss when ps is not in varmap

# varmap (flat): <model variable>: <variable in your files>. The model variable is used as it
# is when a file has a variable of that name; list the others here. Keys:
# climate (REQUIRED, derived to TEM units):  tair or tmin + tmax;  dtr or tmin + tmax;  prec;
# nirr;
#       vpr or dewpoint or huss (+ ps)
#   optional from the climate files:  wind or u10 + v10;  elev
#   passed to the model for the other files:  texture (sand+silt+clay), lulc, co2, ozone,
#       ndep, wind, elev, lightning, popdens, nfertcrop, nfertpasture, mask
varmap: {}
```

5. Model options

```yaml
# Initialization and equilibrium
# 0: equilibrate to long-term average climate | 1: then numspin spins | 2: dynamic equilibrium
spinflag: 0
numspin: 3 # Spins when spinflag is 1
spintime: 40 # Years per spin
de_nyears: 30 # Years of climate used in dynamic equilibration
equil_min_years: 500 # Years before the first equilibrium check
equil_max_years: 3000 # Upper limit of years to reach equilibrium
equil_max_attempts: 1 # Restarts of the equilibration of a cell
equil_tau_years: 500.0 # Averaging time of the equilibrium test
equil_rh: 1 # 1: decomposition must reach equilibrium
wtol: 1.0 # Tolerance of the water cycle
ctol: 100.0 # Tolerance of the carbon cycle
ntol: 20.0 # Tolerance of the nitrogen cycle

# Processes
avlnflag: 1 # Available N may vary
nfeed: 1 # N feedback on GPP
baseline: 0 # Solve for baseline soil N
moistlim: 1 # Moisture limitation of GPP
o3flag: 1 # Ozone damage to GPP
ode_solver: 1 # 1 Bogacki-Shampine | 2 Runge-Kutta-Fehlberg
rsoil_method: 2 # 1 Q10 | 2 SiB
rhmoist_method: 1 # 1 WFPS | 2 DNDC | 3 sech
wood_pools: 0 # 0: standing dead only | 1: GDSTEM-FIRE standing dead, CWD and FWD pools
dc2n: 0.0 # Relative change of C:N per change in CO2
inittol: 0.01 # Proportional tolerance of the integrator
maxit: 20 # Iterations of the integrator per month
maxitmon: 200 # Times maxit can be reached in a month

# Atmosphere, fire and fertilizer
initco2: 286.596 # ppmv at the start
xco2level: 286.596 # ppmv of the equilibrium
xtco2flag: 1 # 1: transient CO2 from netcdf_co2_in_file | 0: static
xto3flag: 0 # 1: transient ozone | 0: long-term average
xtndepflag: 0 # 1: transient N deposition | 0: long-term average
xtws10flag: 0 # 1: transient wind | 0: long-term average
# 0: built-in fertilizer series | 1: annual input without time | 2: transient input
xnfertflag: 0
# 1: wildfire; needs netcdf_fire_in_file, netcdf_population_in_file and monthly steps
veg_fire: 0
storm_disturbance: false # Enable transient storm and hurricane damage. Needs a STORM=1 build.
fire_mode: 0 # 0: random draw | 1: expected burn
fire_topography: 0 # 1: fire 250 times less likely above 2500 m
fire_seed: 0 # Fixes the random sequence (0: not fixed)
fire_prescribed_end: 2014 # Last year in which the vegetation return interval applies
tlulcflag: 0 # 1: transient land use

# Agriculture
agtillflag: 0 # Tilling
agirrgflag: 0 # Irrigation
agfertflag: 0 # Fertilizer
```

6. Parameter files

```yaml
# One set of params/ per run: biogeography, gdstem, gdstem_fire or glstem.

cmntecd: "{tem_src}/params/biogeography/tcomm50_lulc.yaml" # Vegetation mosaics
rootzecd: "{tem_src}/params/biogeography/trotz50_lulc.yaml" # Rooting depth
vegecd: "{tem_src}/params/biogeography/tveg50_lulc.yaml" # Vegetation
conductecd: "{tem_src}/params/biogeography/tconduct50_lulc.yaml" # Hydraulic conductance
mcrbecd: "{tem_src}/params/biogeography/tmcrv50_lulc.yaml" # Microbial
agecd: "{tem_src}/params/biogeography/ag50_lulc.yaml" # Agriculture
# Site calibrations use top-level *_dat keys. Each file declares its TEM community ID.
ice_dat: "{tem_src}/params/biogeography/ice50_openn_303.yaml"
tpd_dat: "{tem_src}/params/biogeography/tpd50_openn_303.yaml"
bnz_dat: "{tem_src}/params/biogeography/bnz50_openn_303.yaml"
hvc_dat: "{tem_src}/params/biogeography/hvc50_openn_dndc.yaml"
hvd_dat: "{tem_src}/params/biogeography/hvd50_openn_dndc.yaml"
knz_ag0_dat: "{tem_src}/params/biogeography/knz50_openn_303_ag0.yaml"
pwn_dat: "{tem_src}/params/biogeography/pwn50_openn_303.yaml"
cur_dat: "{tem_src}/params/biogeography/cur50_openn_303.yaml"
duc_dat: "{tem_src}/params/biogeography/duc50_openn_303.yaml"
gux_dat: "{tem_src}/params/biogeography/gux50_openn_303.yaml"
tai_dat: "{tem_src}/params/biogeography/tai50_openn_303.yaml"
gum_dat: "{tem_src}/params/biogeography/gum50_openn_303.yaml"
kbm_dat: "{tem_src}/params/biogeography/kbm50_openn_303.yaml"
turf_dat: "{tem_src}/params/biogeography/turf50_openn_303.yaml"
knz_ag2_dat: "{tem_src}/params/biogeography/knz50_openn_303_ag2.yaml"
rodale_ag2_dat: "{tem_src}/params/biogeography/rodale_openn_303_ag2.yaml"
```

7. Output and model state

```yaml
output_file: ./OUTPUT/tem.nc # Native TEM NetCDF output, relative to the case directory
clmoutvars: [tair] # Climate variables written (lowercase model names)
temoutvars: [gpp, npp, nep, vegc, soilorgc] # TEM variables written
spinoutflag: 0 # 1: also write the spin-up years
spinoutyrs: 0 # Number of spin-up years written
# 0: initialize | 1: read the state at equilibrium | 2: read the state of istateyear
istateflag: 0
istatefile: ./OUTPUT/temstate.nc # State file read
istateyear: missing # Year of the state read when istateflag is 2
ostateflag: 0 # 0: no state written | 1: at equilibrium | 2: at ostateyear
ostatefile: ./OUTPUT/temstate.nc # State file written (ranks are merged into it)
ostateyear: missing # Year of the state written when ostateflag is 2
```